

KuchenApp

Projektdokumentation

Desktop-Anwendung zur Verwaltung von
Kuchen-Listen und IServ-Mail-Versand

Erstellt mit PySide6 (Qt for Python)

Python 3.10+

17. April 2026

Inhaltsverzeichnis

1	Projektübersicht	2
1.1	Funktionsumfang	2
1.2	Technologie-Stack	2
2	Projektstruktur	2
2.1	Verzeichnisstruktur	2
2.2	Trennung: develop vs. release	3
3	Architektur und Entwurfsmuster	3
3.1	Gesamtarchitektur	3
3.2	Verwendete Entwurfsmuster	4
3.2.1	Singleton-Pattern (AppLogger)	4
3.2.2	Observer-Pattern (Qt Signal/Slot)	4
3.2.3	Model-View-Pattern (Qt MVC)	5
4	Module im Detail	5
4.1	KuchenMain.py – Hauptfenster	5
4.2	DataManager.py – Datenverwaltung	6
4.3	SettingsManager.py – Einstellungsverwaltung	7
4.4	AppLogger.py – Singleton-Logger	7
4.5	KuchenMailManager.py – Mail-Listen-Dialog	7
4.6	LoginDialog.py – IServ-Login	7
4.7	IServMailManager.py – Mail-Versand	8
4.8	IServAPIEdited_standalone.py – IServ-API	8
5	Datenpersistenz	8
5.1	CSV-Format	8
5.2	Settings (INI)	8
6	Build-System	9
6.1	build_release.py	9
6.2	PyInstaller – Standalone-Builds	9
6.3	GitHub Actions – CI/CD	10
7	Sicherheitsaspekte	10
8	Benutzerhandbuch	10
8.1	Installation (Source-Version)	10
8.2	Installation (Standalone)	10
8.3	Bedienung	10
9	Glossar	11

1 Projektübersicht

KuchenApp ist eine Desktop-Anwendung, die im Rahmen eines Schulprojekts (Berufsschule, Fachinformatiker Anwendungsentwicklung) entwickelt wurde. Die Anwendung dient zur Verwaltung einer Kuchen-Liste für eine Schulklasse und ermöglicht den automatisierten Versand von Erinnerungs-E-Mails über das IServ-System.

1.1 Funktionsumfang

- CSV-Dateien importieren, bearbeiten und speichern
- Dynamische Tabellendarstellung mit Sortier- und Suchfunktion
- Aufsteigende und absteigende Sortierung per Radio-Buttons
- Verwaltung einer Empfänger-Mailliste (separater Dialog)
- Automatische HTML-Tabellengenerierung für E-Mail-Inhalte
- E-Mail-Versand über IServ-SMTP mit Login-Dialog
- INI-basierte Einstellungsverwaltung
- Standalone-Build für Linux und Windows (PyInstaller)
- CI/CD über GitHub Actions

1.2 Technologie-Stack

Komponente	Technologie
Programmiersprache	Python 3.10+
GUI-Framework	PySide6 (Qt 6 for Python)
UI-Design	Qt Designer (.ui-Dateien)
Datenpersistenz	CSV (comma-separated values)
Einstellungen	INI-Dateien (configparser)
E-Mail-Versand	smtpplib (SMTP_SSL)
Build/Packaging	PyInstaller
CI/CD	GitHub Actions

Tabelle 1: Verwendete Technologien

2 Projektstruktur

Das Projekt folgt einer klaren Trennung zwischen Entwicklungs- und Release-Code.

2.1 Verzeichnisstruktur

```

1 Kuchen/
2 |-- build_release.py           # Build-Skript
3 |-- requirements.txt          # Python-Abhängigkeiten
4 |-- .github/workflows/        # CI/CD Pipeline

```

```

5 |   +-- build_release.yml
6 | -- develop/                # Entwicklungsumgebung
7 |   |-- KuchenMain.py       # Hauptprogramm (Entry Point)
8 |   |-- src/                 # Quellcode-Module
9 |   |   |-- __init__.py
10 |   |   |-- AppLogger.py
11 |   |   |-- DataManager.py
12 |   |   |-- IServAPIEdited_standalone.py
13 |   |   |-- IservMailManager.py
14 |   |   |-- KuchenMailManager.py
15 |   |   |-- LoginDialog.py
16 |   |   +-- SettingsManager.py
17 |   |-- ui/                  # UI-Dateien
18 |   |   |-- __init__.py
19 |   |   |-- UIMainWindow.ui / _ui.py
20 |   |   |-- UIMailsDialog.ui / _ui.py
21 |   |   +-- UILoginDialog.ui / _ui.py
22 |   |-- assets/
23 |   |   +-- kuchen_icon.svg
24 |   +-- Data/
25 |       |-- CakeData.csv
26 |       |-- StandardCakeData.csv
27 |       |-- Settings/settings.ini
28 |       +-- Klassen/EITB23A.csv
29 +-- release/                # Generierte Releases
30 |   |-- KuchenApp/           # Source-Bundle
31 |   |-- KuchenApp.zip
32 |   |-- KuchenApp_standalone_linux/
33 |   |-- KuchenApp_standalone_linux.zip
34 |   |-- KuchenApp_standalone_windows/
35 |   +-- KuchenApp_standalone_windows.zip

```

Listing 1: Gesamtstruktur

2.2 Trennung: develop vs. release

develop/ Enthält den vollständigen Quellcode, .ui-Designdateien und Testdaten. Hier wird aktiv entwickelt.

release/ Wird durch `build_release.py` automatisch generiert. Enthält nur die für den Endbenutzer nötigen Dateien. Die CSV-Dateien werden mit leeren Standard-Headern ausgeliefert.

3 Architektur und Entwurfsmuster

3.1 Gesamtarchitektur

Die Anwendung folgt einer **Schichtenarchitektur**:

1. **Präsentationsschicht** – GUI-Klassen (CMainWindow, CKuchenDialog, CLoginDialog), Qt-UI-Dateien

2. **Logikschicht** – Datenverarbeitung (DataManager), Einstellungen (SettingsManager), Mail-Versand (IServMailManager)
3. **Datenzugriffsschicht** – CSV-Dateien, INI-Konfiguration, IServ-SMTP-API

3.2 Verwendete Entwurfsmuster

3.2.1 Singleton-Pattern (AppLogger)

Der AppLogger implementiert das **Singleton-Pattern** über Pythons `__new__`-Methode. Damit existiert während der gesamten Laufzeit exakt eine Logger-Instanz.

Fachliche Begründung: Alle Klassen sollen Fehlermeldungen an die GUI weiterleiten können, ohne eine Referenz auf das Hauptfenster zu benötigen. Der Singleton stellt sicher, dass alle `AppLogger()`-Aufrufe dieselbe Instanz zurückgeben und somit die gleichen Qt-Signale nutzen.

```
1 class AppLogger(QObject):
2     errorOccurred = Signal(str, str)
3     warningOccurred = Signal(str, str)
4     infoOccurred = Signal(str, str)
5
6     _instance = None
7
8     def __new__(cls):
9         if cls._instance is None:
10             cls._instance = super().__new__(cls)
11         return cls._instance
12
13     def __init__(self):
14         if not hasattr(self, '_initialized'):
15             super().__init__()
16             self._initialized = True
```

Listing 2: Singleton-Implementierung

3.2.2 Observer-Pattern (Qt Signal/Slot)

Die Kommunikation zwischen den Schichten erfolgt über das **Observer-Pattern**, implementiert durch Qt's Signal/Slot-Mechanismus.

- AppLogger emittiert Signale (`errorOccurred`, `warningOccurred`, `infoOccurred`)
- CMainWindow verbindet diese mit Slot-Methoden, die QMessageBox-Dialoge anzeigen
- QStandardItemModel.dataChanged benachrichtigt bei Zellenänderungen in der Tabelle

Fachliche Begründung: Lose Kopplung – der DataManager und IServMailManager kennen die GUI nicht, sondern kommunizieren über den AppLogger. Dies ermöglicht eine saubere Trennung der Schichten.

3.2.3 Model-View-Pattern (Qt MVC)

Die Tabellendarstellung nutzt Qt's **Model-View**-Architektur:

- **Model:** `QStandardItemModel` – hält die Tabelleneinträge
- **View:** `QTableView` – zeigt die Daten an
- **Controller:** Die jeweilige Window-Klasse vermittelt zwischen `DataManager` und dem Model

4 Module im Detail

4.1 KuchenMain.py – Hauptfenster

`CMainWindow` erbt von `QMainWindow` und `Ui_MainWindow` (Multiple Inheritance, empfohlener Qt-Ansatz). Es ist der **Entry Point** der Anwendung.

Schlüsselfunktionen:

`_createRadioButtons()`

Erzeugt dynamisch Radio-Buttons für jede CSV-Spalte (Sortierung) sowie ein Ascending/Descending Paar. Verwendet `QButtonGroup` für die Sortierrichtung.

`SortColumn()`

Liest den ausgewählten Radio-Button aus, bestimmt den Spaltenindex und ruft `DataManager.sortData()` mit dem `aReverse`-Parameter auf. Berücksichtigt aktive Suchfilter.

`Search(aText)`

Filtert die Originaldaten (`mData`) nach dem Suchbegriff. Ist ein Sortier-Radio-Button aktiv, wird nur in der entsprechenden Spalte gesucht. Ansonsten: Volltextsuche über alle Spalten. Nach dem Filtern wird die Sortierung erneut angewandt.

`CellEdited(aItem)`

Synchronisiert Änderungen in der Tabelle zurück in `mSortedData` und `mData`. Sucht die Original-Zeile anhand aller anderen Spaltenwerte (da die Reihenfolge abweichen kann).

`FillTable()`

Löscht das `QStandardItemModel`, setzt Header und füllt die Tabelle aus `mSortedData`. Wird nach jeder Datenänderung aufgerufen.

`SendMails()`

Zeigt einen Bestätigungsdialog mit der Empfängeranzahl, öffnet den Login-Dialog und wertet den Tri-State-Rückgabewert aus (`None`=abgebrochen, `True`=gesendet, `False`=fehlgeschlagen).

`ImportFile()`

Öffnet einen `QFileDialog`, importiert die CSV und aktualisiert die Radio-Buttons dynamisch anhand der neuen Header.

PyInstaller-Kompatibilität: Im `__main__`-Block wird `sys.frozen` geprüft, um den

Icon-Pfad korrekt aufzulösen (`sys._MEIPASS` im Bundle vs. `__file__` im Entwicklungsmodus).

4.2 DataManager.py – Datenverwaltung

Der `DataManager` ist die zentrale Klasse für alle Datenoperationen. Er verwaltet zwei unabhängige Datensätze:

- **Haupttabelle** (Kuchen-Daten): `mData`, `mSortedData`, `mMainHeaders`
- **Mail-Tabelle** (Empfänger): `mMailData`, `mSortedMailData`, `mMailHeaders`

Duale Datenhaltung: `mData` enthält immer die *unsortierte* Originaldaten. `mSortedData` ist eine gefilterte/sortierte Kopie, die in der Tabelle angezeigt wird. Dies ermöglicht das Zurücksetzen von Sortierung und Suche.

Schlüsselfunktionen:

`LoadStandardFile(aForMainTable)`

Lädt die CSV-Datei, deren Name in den Settings steht. Verwendet `csv.Sniffer` zur automatischen Dialekt-Erkennung (Trennzeichen, Quoting). Extrahiert die erste Zeile als Header.

`ImportFile(aPath)`

Importiert eine externe CSV. Speichert den Dateinamen in `mImportedFilename`. Beim nächsten `SaveFile()` wird dieser Name in die Settings übernommen, sodass die importierte Datei beim nächsten Start automatisch geladen wird.

`SaveFile(aForMainTable)`

Speichert die Daten als CSV. Wurde zuvor eine Datei importiert, aktualisiert `SaveFile` automatisch die Settings mit dem neuen Dateinamen.

`sortData(aSortColumnIndex, aReverse, aForMain)`

Sortiert `mSortedData` (oder `mSortedMailData`) nach der gegebenen Spalte. Der Parameter `aReverse` steuert die Richtung (aufsteigend/absteigend).

`_padRows(aData, aHeaderLen)`

Füllt Datenzeilen mit leeren Strings auf, falls sie weniger Spalten als der Header haben. Dies erhöht die Robustheit beim Import unvollständiger CSVs.

`createCompleteMailData()`

Bereitet die Mail-Daten auf: extrahiert E-Mail-Adressen und generiert einen HTML-String der Kuchen-Tabelle für den E-Mail-Body.

`createHtmlDataString()`

Erzeugt eine formatierte HTML-Tabelle mit Inline-CSS. Verwendet `html.escape()` zur Vermeidung von XSS-Schwachstellen.

PyInstaller-Kompatibilität: Die Funktion `_get_base_dir()` gibt bei `sys.frozen == True` das Verzeichnis der ausführbaren Datei zurück, ansonsten das Projektverzeichnis.

4.3 SettingsManager.py – Einstellungsverwaltung

Verwaltet die Anwendungseinstellungen in einer INI-Datei (Data/Settings/settings.ini) über Pythons configparser-Modul.

Konfigurationsstruktur:

```
1 [Klassen]
2 Dateiname = StandardKlasse.csv
3
4 [CakeData]
5 Dateiname = StandardCakeData.csv
6
7 [IServ]
8 Domain = wvss.de
```

Listing 3: settings.ini

Default-Werte: Die Klasse definiert ein `_DEFAULTS`-Dictionary. Beim Instanziiieren werden zuerst die Defaults geladen, dann die INI-Datei darübergerlegt. So sind immer Fallback-Werte vorhanden.

4.4 AppLogger.py – Singleton-Logger

Implementiert das Singleton-Pattern (siehe Abschnitt 3.2.1). Stellt drei Schweregrade bereit:

- `error(aTitle, aMessage)` – Kritische Fehler (roter Dialog)
- `warning(aTitle, aMessage)` – Warnungen (gelber Dialog)
- `info(aTitle, aMessage)` – Informationen (blauer Dialog)

Jeder Aufruf emittiert ein Qt-Signal mit Titel und Nachricht. Die GUI verbindet diese Signale mit `QMessageBox.critical()`, `QMessageBox.warning()` bzw. `QMessageBox.information()`.

4.5 KuchenMailManager.py – Mail-Listen-Dialog

`CKuchenDialog` ist ein modaler Dialog (`QDialog`) zur Verwaltung der Empfänger-Mailliste. Er teilt sich den `DataManager` mit dem Hauptfenster und arbeitet auf den Mail-Daten (`mMailData`).

Die Funktionalität ist analog zum Hauptfenster: dynamische Radio-Buttons, Sortierung (ascending/descending), Suche, Import, Speichern, Zeilen hinzufügen/löschen.

4.6 LoginDialog.py – IServ-Login

`CLoginDialog` ist ein modaler Dialog für die SMTP-Authentifizierung.

Tri-State-Logik:

- `mSendState = None` – Benutzer hat abgebrochen
- `mSendState = True` – Mails erfolgreich gesendet
- `mSendState = False` – Fehler beim Senden (Dialog bleibt offen)

4.7 IServMailManager.py – Mail-Versand

Erstellt eine Verbindung zur IServ-API und sendet E-Mails über SMTP_SSL. Das Passwort wird im `finally`-Block auf `None` gesetzt, um es nicht unnötig im Speicher zu halten (**Security Best Practice**).

Fehlerbehandlung fängt spezifische Exceptions: `OSError`, `smtplib.SMTPException`, `ValueError`, `ConnectionError`.

4.8 IServAPIEdited_standalone.py – IServ-API

Eine angepasste Version der IServ-API, die **ausschließlich** die Python-Standardbibliothek verwendet (kein `pip install` nötig).

Ersetzt externe Abhängigkeiten:

- `requests` → `urllib.request`
- `beautifulsoup4` → eigene `_HTMLNode`-Klasse
- `smtplib` für SMTP_SSL-Verbindungen

Für den E-Mail-Versand wird `email.mime` zur MIME-Nachrichtenerstellung und `smtplib.SMTP_SSL` für die verschlüsselte Verbindung genutzt.

5 Datenpersistenz

5.1 CSV-Format

Die Anwendung speichert Daten als CSV-Dateien (Comma-Separated Values). Beim Laden wird `csv.Sniffer` zur automatischen Erkennung des Trennzeichens verwendet – so werden sowohl Komma- als auch Semikolon-getrennte Dateien korrekt gelesen.

StandardCakeData.csv (leere Vorlage):

```
1 Name , CakeCount , Hanuta , Waffel , Date
```

Listing 4: CSV-Header

Dynamische Header: Die Header werden nicht hartcodiert, sondern aus der ersten Zeile der CSV-Datei gelesen. Die GUI-Radio-Buttons für die Sortierung werden dynamisch aus diesen Headern generiert.

5.2 Settings (INI)

Einstellungen werden über Pythons `configparser`-Modul in einer INI-Datei verwaltet. Die Datei wird mit Defaults initialisiert und bei Änderungen (z. B. Import einer neuen CSV) automatisch aktualisiert.

6 Build-System

6.1 build_release.py

Das zentrale Build-Skript automatisiert die Release-Erstellung:

```
1 python build_release.py           # Alle 3 Versionen
2 python build_release.py --source  # Nur Source-Bundle
3 python build_release.py --linux   # Nur Linux Standalone
4 python build_release.py --windows # Nur Windows Standalone
```

Listing 5: Verwendung

Build-Schritte:

1. **Clean:** Alter release/-Ordner wird gelöscht
2. **UI-Kompilierung:** .ui → _ui.py via pyside6-uic
3. **Kopieren:** Quelldateien werden in die Release-Struktur kopiert
4. **README:** Automatische README.txt-Generierung
5. **ZIP:** Source-Bundle als ZIP verpackt
6. **PyInstaller:** Standalone-Binary für das aktuelle OS
7. **Remote-Build:** Für das andere OS via GitHub Actions (mit gh CLI)

6.2 PyInstaller – Standalone-Builds

PyInstaller verpackt Python-Interpreter, PySide6 und alle Quelldateien in eine einzelne ausführbare Datei (-onefile).

Wichtige Optionen:

- -onefile – Einzelne Datei statt Ordner
- -windowed – Kein Konsolenfenster (GUI-App)
- -add-data – Icon und Data-Verzeichnis einbetten

Laufzeit-Pfadauflösung: Da PyInstaller Dateien in ein temporäres Verzeichnis (`sys._MEIPASS`) entpackt, aber Data/ beschreibbar sein muss, wird Data/ neben die Executable kopiert. Die Funktion `_get_base_dir()` in `DataManager` und `SettingsManager` unterscheidet automatisch:

```
1 def _get_base_dir():
2     if getattr(sys, 'frozen', False):
3         return os.path.dirname(sys.executable)
4     return os.path.dirname(os.path.dirname(
5         os.path.abspath(__file__)))
```

Listing 6: Pfadauflösung

6.3 GitHub Actions – CI/CD

Der Workflow `.github/workflows/build_release.yml` baut automatisch für beide Betriebssysteme:

- **Trigger:** Version-Tag (`v*`) oder manueller Dispatch
- **Matrix-Build:** Ubuntu + Windows parallel
- **Release-Job:** Erstellt ein GitHub Release mit allen 3 ZIPs als Download-Anhänge

```
1 git tag v1.0
2 git push --tags
```

Listing 7: Release auslösen

7 Sicherheitsaspekte

- **HTML-Escaping:** Alle Tabelleninhalte werden mit `html.escape()` in der Mail-HTML-Generierung escaped, um Cross-Site-Scripting (XSS) zu verhindern.
- **Passwort-Löschung:** Das SMTP-Passwort wird im `finally`-Block von `IserveMailManager.send` auf `None` gesetzt.
- **Spezifische Exceptions:** Statt generischem `except Exception` werden nur erwartete Exception-Typen gefangen (`OSError`, `csv.Error`, `smtpplib.SMTPException`, etc.).
- **SMTP_SSL:** Die Verbindung zu IServ erfolgt über SSL-verschlüsseltes SMTP (Port 465).

8 Benutzerhandbuch

8.1 Installation (Source-Version)

```
1 pip install PySide6
2 cd KuchenApp
3 python KuchenMain.py
```

8.2 Installation (Standalone)

Die Standalone-Version benötigt keine Installation:

- **Linux:** `./KuchenApp`
- **Windows:** Doppelklick auf `KuchenApp.exe`

8.3 Bedienung

1. **Daten laden:** Beim Start wird die zuletzt verwendete CSV automatisch geladen.
2. **Import:** Über den *Import*-Button kann eine andere CSV-Datei geladen werden. Die Spalten und Sortier-Buttons passen sich automatisch an.

3. **Bearbeiten:** Zellen können direkt in der Tabelle bearbeitet werden (Doppelklick).
4. **Zeilen hinzufügen/löschen:** Über die Buttons *Add New* und *Delete*.
5. **Sortierung:** Radio-Buttons rechts neben der Tabelle. Ascending/Descending wählt die Richtung.
6. **Suche:** Das Suchfeld filtert live. Bei aktivem Sortier-Button wird nur in der entsprechenden Spalte gesucht.
7. **Speichern:** Über den *Save*-Button. Bei importierten Dateien wird der neue Dateiname in den Settings gespeichert.
8. **Mail-Liste:** Über *View/Edit Mail List* öffnet sich ein separater Dialog zur Verwaltung der Empfänger.
9. **Mails senden:** Über *SendMails*. Es erscheint eine Bestätigung mit der Empfängeranzahl, dann der Login-Dialog.

9 Glossar

CSV Comma-Separated Values – Tabellenformat mit Trennzeichen

INI Initialisierungsdatei – Einfaches Konfigurationsformat

PySide6 Offizielle Python-Bindings für das Qt 6 Framework

Qt Designer Visueller Editor für Qt-Oberflächen (.ui-Dateien)

PyInstaller Tool zum Verpacken von Python-Programmen als Standalone

SMTP_SSL Simple Mail Transfer Protocol über SSL-Verschlüsselung

Singleton Entwurfsmuster: Garantiert genau eine Instanz einer Klasse

Observer Entwurfsmuster: Objekte werden über Zustandsänderungen benachrichtigt (in Qt: Signal/Slot)

MVC Model-View-Controller – Architekturmuster zur Trennung von Daten, Darstellung und Steuerung

CI/CD Continuous Integration / Continuous Deployment – Automatisierte Build- und Release-Pipelines

XSS Cross-Site Scripting – Sicherheitslücke durch unescapten HTML-Input

IServ Schulplattform mit E-Mail, Kalender und Dateiverwaltung